SECTION 3: DATA ARCHITECTURE — WHERE STATE LIVES, DIES, AND LIES ⏱️ 9 min • MODULE 12

# Part 12: Caching Strategies: The Two Hard Things in Computer Science

*How to turbocharge throughput without poisoning your system with stale or phantom data.*

🔥 **WAR STORY FROM THE TRENCHES**

A media streaming website cached user subscription entitlements in Redis for 24 hours to reduce database read load. A user upgraded from the Free tier to the Premium VIP plan. Her credit card was billed $19.99 instantly. But when she tried to watch the live championship boxing match, the app displayed: 'Upgrade your subscription to watch this event.' She clicked upgrade again and was billed a second time. Enraged, she tweeted a screenshot of her double charge. Over 40,000 subscribers suffered the same bug during the fight. Why? The billing service updated the SQL database, but the caching layer had no cache invalidation trigger—it simply waited for the 24-hour TTL to expire!

## The Paradigm Shift: From Coder to Architect Mindset

Phil Karlton's legendary adage has haunted architects for decades: 'There are only two hard things in Computer Science: cache invalidation and naming things.' Caching is not a magic speed button; it is an explicit decision to maintain two copies of the same truth. The moment you introduce a cache, you enter the dangerous world of cache coherence, stale data, and stampedes.

## Core Architectural Principles & Mental Models

### The Core Caching Topologies

- **Cache-Aside (Lazy Loading):** Application queries the cache. On miss, it reads from the DB, writes to the cache, and returns. Simple, robust, but initial reads pay miss latency.
- **Write-Through:** Application writes to cache, and cache immediately writes to DB synchronously. Ensures cache is always fresh, but write latency is higher.
- **Write-Behind (Write-Back):** Application writes to cache, which acknowledges immediately and writes to DB asynchronously. Extreme write speed, but data can be lost if cache crashes before persistence.
- **Refresh-Ahead:** System automatically reloads hot cache keys before they expire based on access frequency.

### The Thundering Herd (Cache Stampede)

When a hot cache key accessed by 10,000 requests/second expires at exactly 12:00:00, all 10,000 requests experience a cache miss simultaneously and hit the database at the same millisecond, crashing the database. Solution: Mutex locks on cache miss (SingleFlight) or probabilistic early expiration (XFetch).

## Common Anti-Patterns & Traps to Avoid

- **The Missing Invalidation Hook:** Updating a record in the database without deleting or updating the corresponding key in Redis.
- **The Infinite Cache:** Setting no TTL (Time-To-Live) on cache keys, causing Redis memory to grow until it crashes from Out-Of-Memory (OOM) or triggers aggressive LRU eviction.

- **Caching Business Logic:** Putting complex calculated objects into cache that depend on 10 different entities, making invalidation practically impossible.

> 🎯 **THE ARCHITECT'S PLAYBOOK: ACTION STEPS THIS WEEK**
>
> - **Step 1:** Always set an explicit TTL on every single cache key. Never store anything in Redis without an expiration.
> - **Step 2:** Prefer cache eviction (`DEL key`) over cache updating on write. Let the next read cleanly reload from source of truth.
> - **Step 3:** Add jitter to your TTLs (`TTL = 300s + rand(0, 60s)`) to prevent multiple hot keys from expiring simultaneously.

## Architectural Takeaways & Cheatsheet

- A cache is a performance booster, not a primary datastore. If your system dies when the cache is flushed, your architecture is broken.
- Eviction on write is safer and cleaner than mutation on write.
- Always add random jitter to TTLs to prevent the catastrophic cache stampede.

> 🎙️ **UDEMY & NOTEBOOKLM DISCUSSION PROMPT**
>
> **Course Reflection & Hands-on Assignment:** Audit your Redis or Memcached instance: Are there keys without TTLs? What happens to your primary database if your cache cluster is suddenly restarted with an empty memory?